SERVER TẠO HTML NHƯ THẾ NÀO?

MSc. Ngô Ngọc Đăng Khoa

SERVER TẠO HTML NHƯ THẾ NÀO?

NỘI DUNG BUỔI HỌC

01 Câu hỏi và mental model

02 Khái niệm và ví dụ

03 Bài tập

04 Nghiệm thu và trình bày

05 Tổng kết và đối chiếu

CÂU HỎI LỚN

> Nếu HTML không phải file có sẵn mà được tạo từ dữ liệu thì server phải làm gì?

MỤC TIÊU ĐẦU RA

Theo dõi luồng **REQUEST** → **HANDLER** → **TEMPLATE**

01

Phân biệt dữ liệu với HTML được render

02

Xử lý form POST ở phía server

03

Giải thích mẫu chung giữa các framework SSR

04

THUẬT NGỮ CỐT LÕI

THUẬT NGỮ	NGHĨA DÙNG TRONG BLOCK
Route	Quy tắc ghép method và URL
Handler	Hàm xử lý request và chọn response
Template	HTML có vị trí nhận dữ liệu
Template engine	Thư viện biên dịch template và data thành HTML
Render	Kết hợp dữ liệu với template thành HTML

MENTAL MODEL

SO SÁNH

GET /students

- Đọc danh sách từ nguồn dữ liệu
- Render template với mảng student
- Trả HTML hoàn chỉnh cho browser

POST /students

- Đọc và validate form body
- Thêm student nếu hợp lệ
- Redirect về danh sách để tránh resubmit

VÍ DỤ TRỌNG TÂM

```
app.get("/students", (req, res) ⇒ {
  res.render("students", { students });
});

app.post("/students", (req, res) ⇒ {
  students.push({ name: req.body.name });
  res.redirect("/students");
});
```

- GET **đọc dữ liệu** và tạo HTML
- POST **thay đổi dữ liệu** từ form body
- Redirect tạo request GET mới sau submit

TEMPLATE ENGINE

```
<!-- views/students.ejs -->
<ul>
  <% students.forEach(s => { %>
    <li><%= s.name %> – <%= s.email %></li>
  <% }) %>
</ul>
```

- Template engine thay `<%= %>` bằng dữ liệu và **tự escape HTML**
- Template chỉ lặp, rẽ nhánh đơn giản; logic nghiệp vụ ở handler
- `res.render("students", { students })` = chọn template + truyền data

CÙNG PATTERN, NHIỀU STACK

STACK	TEMPLATE ENGINE	CÚ PHÁP LẶP
Node.js + Express	EJS, Handlebars, Pug	`<% for ... %>`
Python + Flask	Jinja2	`{% for s in students %}`
PHP + Laravel	Blade	`@foreach ($students as $s)`
ASP.NET Core	Razor	`@foreach (var s in Model)`
Java + Spring Boot	Thymeleaf	`th:each="s : ${students}"`

LỖI THƯỜNG GẶP

Đặt toàn bộ logic xử lý trong template 01

Không validate dữ liệu từ form 02

POST xong render trực tiếp gây gửi lại form 03

Nhầm route server với file HTML vật lý 04

20–60 PHÚT

BÀI TẬP

Xây Student List dạng server-rendered với route danh sách và form tạo mới.

DỮ LIỆU VÀ ĐIỀU KIỆN BAN ĐẦU

Starter Express và EJS hoặc stack tương đương (Flask + Jinja2, Laravel + Blade...)

01

Dữ liệu file-based: `data/students.json` với ba Student mẫu, hoặc SQLite file

02

Không dùng database server hay ORM trong bài này

03

Student tối thiểu có `id`, `name`, `email`

04

Hai route bắt buộc: GET và POST `/students`

05

YÊU CẦU CHỨC NĂNG

GET `/students` render danh sách bằng template	01
Trang có form Create Student	02
POST `/students` đọc, validate và lưu dữ liệu	03
Sau submit, server render lại danh sách cập nhật	04

DỮ LIỆU?

- **Không dùng database server**; tập trung vào luồng render
- Mức 1: mảng trong bộ nhớ, mất khi restart server
- Mức 2: file `students.json` đọc và ghi bằng `fs`, hoặc `lowdb`
- Mức 3: SQLite một file (`better-sqlite3`, `sqlite3`) nếu muốn SQL
- Dữ liệu mẫu: tự viết 3 Student hoặc nhờ agent sinh `seed.json`

QUY TRÌNH 40 PHÚT

01

ROUTE GET — Chuẩn bị data và render template

02

TEMPLATE — Lặp danh sách, tạo form semantic

03

ROUTE POST — Đọc body, validate và thêm record

04

KIỂM TRA — Submit hợp lệ, rỗng và refresh

TIÊU CHÍ NGHIỆM THU

HTML response chứa dữ liệu từ server

01

Student mới xuất hiện sau submit

02

Input rỗng nhận thông báo phù hợp

03

Refresh không tạo bản ghi trùng

04

BẰNG CHỨNG PHẢI NỘP

Danh sách route và method	01
Ảnh response HTML hoặc View Source	02
Log request GET/POST	03
Mô tả cách agent xử lý redirect	04

TRÌNH BÀY VÀ TỔNG KẾT

Hai nhóm ngẫu nhiên · 10 phút mỗi nhóm

KỊCH BẢN DEMO 10 PHÚT

Mở `/students` và View Source

01

Submit Student hợp lệ

02

Thử dữ liệu rỗng

03

Refresh và giải thích request mới

04

ĐIỀU CẦN NHỚ

SSR tạo **HTML TẠI SERVER** theo request

01

Handler nối request, data và template

02

Form submit tạo request mới tới server

03

Pattern chung tồn tại ngoài cú pháp từng framework

04

Q & A

Nộp artifact trước khi rời lớp